Homework 5: Conditional Probability Models

Instructions: Your answers to the questions below, including plots and mathematical work, should be submitted as a single PDF file. It's preferred that you write your answers using software that typesets mathematics (e.g. LaTeX, LyX, or MathJax via iPython), though if you need to you may scan handwritten work. You may find the minted package convenient for including source code in your LaTeX document. If you are using LyX, then the listings package tends to work better.

1 Introduction

In this homework we'll be investigating conditional probability models, with a focus on various interpretations of logistic regression, with and without regularization. Along the way we'll discuss the calibration of probability predictions, both in the limit of infinite training data and in a more bare-hands way. On the Bayesian side, we'll recreate from scratch the Bayesian linear gaussian regression example we discussed in lecture. We'll also have several optional problems that work through many basic concepts in Bayesian statistics via one of the simplest problems there is: estimating the probability of heads in a coin flip. Later we'll extend this to the probability of estimating click-through rates in mobile advertising. Along the way we'll encounter empirical Bayes and hierarchical models.

2 From Scores to Conditional Probabilities[1]

Let's consider the classification setting, in which $(x_1, y_1), \ldots, (x_n, y_n) \in \mathcal{X} \times \{-1, 1\}$ are sampled i.i.d. from some unknown distribution. For a prediction function $f : \mathcal{X} \to \mathbf{R}$, we define the **margin** on an example (x, y) to be $m = yf(x)$. Since our class predictions are given by $\text{sign}(f(x))$, we see that a prediction is correct iff $m(x) > 0$. It's tempting to interpret the magnitude of the score $|f(x)|$ as a measure of confidence. However, it's hard to interpret the magnitudes beyond saying one prediction score is more or less confident than another, and without any scale to this "confidence score", it's hard to know what to do with it. In this problem, we investigate how we can translate the score into a probability, which is much easier to interpret. In other words, we are looking for a way to convert score function $f(x) \in \mathbf{R}$ into a conditional probability distribution $x \mapsto p(y = 1 \mid x)$.

In this problem we will consider **margin-based losses**, which are loss functions of the form $(y, f(x)) \mapsto \ell(yf(x))$, where $m = yf(x)$ is called the **margin**. We are interested in how we can go from an empirical risk minimizer for a margin-based loss, $\hat{f} = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{n} \ell(y_i f(x_i))$, to a conditional probability estimator $\hat{\pi}(x) \approx p(y = 1 \mid x)$. Our approach will be to try to find a

[1] This problem is based on Section 7.5.3 of Schapire and Freund's book *Boosting: Foundations and Algorithms*.

way to use the Bayes[2] prediction function[3] $f^* = \arg\min_f \mathbb{E}_{x,y}\left[\ell(yf(x)\right]$ to get the true conditional probability $\pi(x) = p(y = 1 \mid x)$, and then apply the same mapping to the empirical risk minimizer. While there is plenty that can go wrong with this "plug-in" approach (primarily, the empirical risk minimizer from a [limited] hypothesis space $\mathcal{F}$ may be a poor estimate for the Bayes prediction function), it is at least well-motivated, and it can work well in practice. And **please note** that we can do better than just hoping for success: if you have enough validation data, you can directly assess how well "calibrated" the predicted probabilities are. This blog post has some discussion of calibration plots: https://jmetzen.github.io/2015-04-14/calibration.html.

It turns out it is straightforward to find the Bayes prediction function f^* for margin losses, at least in terms of the data-generating distribution: For any given $x \in \mathcal{X}$, we'll find the best possible prediction $\hat{y}$. This will be the $\hat{y}$ that minimizes

$$\mathbb{E}_y\left[\ell\left(y\hat{y}\right) \mid x\right].$$

If we can calculate this $\hat{y}$ for all $x \in \mathcal{X}$, then we will have determined $f^*(x)$. We will simply take

$$f^*(x) = \arg\min_{\hat{y}} \mathbb{E}_y\left[\ell\left(y\hat{y}\right) \mid x\right].$$

Below we'll calculate f^* for several loss functions. It will be convenient to let $\pi(x) = p\left(y = 1 \mid x\right)$ in the work below.

1. Write $\mathbb{E}_y\left[\ell\left(yf(x)\right) \mid x\right]$ in terms of $\pi(x)$, $\ell(-f(x))$, and $\ell\left(f(x)\right)$. [Hint: Use the fact that $y \in \{-1, 1\}$.]

   Solution:

   $$\mathbb{E}_y[\ell(yf(x))] = \pi(x)\ell(f(x)) + (1 - \pi(x))\ell(-f(x))$$

2. Show that the Bayes prediction function $f^*(x)$ for the exponential loss function $\ell\left(y, f(x)\right) = e^{-yf(x)}$ is given by

   $$f^*(x) = \frac{1}{2}\ln\left(\frac{\pi(x)}{1 - \pi(x)}\right),$$

   where we've assumed $\pi(x) \in (0, 1)$. Also, show that given the Bayes prediction function f^*, we can recover the conditional probabilities by

   $$\pi(x) = \frac{1}{1 + e^{-2f^*(x)}}.$$

   [Hint: Differentiate the expression in the previous problem with respect to $f(x)$. To make things a little less confusing, and also to write less, you may find it useful to change variables a bit: Fix an $x \in \mathcal{X}$. Then write $p = \pi(x)$ and $\hat{y} = f(x)$. After substituting these into the expression you had for the previous problem, you'll want to find $\hat{y}$ that minimizes the

[2] Don't be confused – it's Bayes as in "Bayes optimal", as we discussed at the beginning of the course, not Bayesian as we've discussed more recently.

[3] In this context, the Bayes prediction function is often referred to as the "population minimizer." In our case, "population" refers to the fact that we are minimizing with respect to the true distribution, rather than a sample. The term "population" arises from the context where we are using a sample to approximate some statistic of an entire population (e.g. a population of people or trees).

expression. Use differential calculus. Once you've done it for a single x, it's easy to write the solution as a function of x.]

Solution:

Let $\ell(y, f(x)) = e^{-yf(x)}, p = \pi(x), \hat{y} = f(x)$

$$\mathbb{E}_y[\ell(y, f(x))] = \underbrace{pe^{-\hat{y}}}_{y=1} + \underbrace{(1-p)e^{\hat{y}}}_{y=-1}$$

$$\frac{\partial}{\partial \hat{y}}(\mathbb{E}_y[\ell(y, f(x))]) = pe^{-\hat{y}}(-1) + e^{\hat{y}} - pe^{\hat{y}} = 0$$

$$= \frac{-p}{e^{\hat{y}}} + e^{\hat{y}} - pe^{\hat{y}} = 0$$

$$\frac{-p}{e^{2\hat{y}}} - p = -1$$

Solving for p we get

$$p = \frac{e^{2\hat{y}}}{1 + e^{2\hat{y}}} = \frac{1}{1 + e^{-2\hat{y}}} = \pi(x)$$

Solving for Bayes prediction function $f^*(x)$,

$$\frac{1}{p} - 1 = \frac{1}{e^{2\hat{y}}}$$

$$e^{2\hat{y}} = \frac{p}{1-p}$$

$$2\hat{y} = \ln\left(\frac{p}{1-p}\right)$$

$$\hat{y} = \frac{1}{2}\ln\left(\frac{\pi(x)}{1-\pi(x)}\right) = f^*(x)$$

3. Show that the Bayes prediction function $f^*(x)$ for the logistic loss function $\ell(y, f(x)) = \ln\left(1 + e^{-yf(x)}\right)$ is given by

$$f^*(x) = \ln\left(\frac{\pi(x)}{1-\pi(x)}\right)$$

and the conditional probabilities are given by

$$\pi(x) = \frac{1}{1 + e^{-f^*(x)}}.$$

Again, we may assume that $\pi(x) \in (0, 1)$.

Solution:

Let $\ell(y, f(x)) = \ln 1 + e^{-y\hat{y}}$

$$\mathbb{E}_y[\ell(y, f(x))] = \underbrace{p \ln 1 + e^{-\hat{y}}}_{y=1} + \underbrace{(1-p)\ln 1 + e^{\hat{y}}}_{y=-1}$$

$$\frac{\partial}{\partial \hat{y}}(\mathbb{E}_y[\ell(y, f(x))]) = p\left(\frac{1}{1+e^{\hat{y}}}\right)e^{-\hat{y}}(-1) + (1-p)\frac{1}{1+e^{\hat{y}}}e^{\hat{y}} = 0$$

$$-\frac{pe^{-\hat{y}}}{1+e^{-\hat{y}}}+\frac{e^{\hat{y}}}{1+e^{\hat{y}}}-\frac{pe^{\hat{y}}}{1+e^{\hat{y}}}=0$$

$$p(\frac{e^{-\hat{y}}}{1+e^{-\hat{y}}}+\frac{e^{\hat{y}}}{1+e^{\hat{y}}})\frac{1+e^{\hat{y}}}{e^{\hat{y}}}=1$$

$$\frac{1}{p}=\frac{e^{-2\hat{y}}(1+e^{\hat{y}})}{1+e^{-\hat{y}}}+1$$

Solving for p we get,

$$p=\frac{1}{1+e^{-\hat{y}}}=\pi(x)$$

and the Bayes prediction function is

$$1+e^{-\hat{y}}=\frac{1}{p}$$

$$\hat{y}=-\ln\frac{1-p}{p}=\ln\frac{p}{1-p}=\ln\frac{\pi(x)}{1-\pi(x)}$$

4. [Optional] Show that the Bayes prediction function $f^*(x)$ for the hinge loss function $\ell\left(y, f(x)\right)=\max\left(0, 1-yf(x)\right)$ is given by

$$f^*(x)=\operatorname{sign}\left(\pi(x)-\frac{1}{2}\right).$$

Note that it is impossible to recover $\pi(x)$ from $f^*(x)$ in this scenario. However, in practice we work with an empirical risk minimizer, from which we may still be able to recover a reasonable estimate for $\pi(x)$. An early approach to this problem is known as "Platt scaling": https://en.wikipedia.org/wiki/Platt_scaling.

Solution:

Let $\ell(y,\hat{y})=\max(0,1-y\hat{y})$

$$\begin{aligned}\mathbb{E}[\ell(yf(x))|x]&=\max(0,1-f(x))\pi(x)+\max(0,1+f(x))(1-\pi(x))\\&=\begin{cases}(1-f(x))\pi(x) & ,f(x)<-1\\(1-f(x))\pi(x)+(1+f(x))(1-\pi(x)) & ,-1<f(x)\leq 1\\(1+f(x))(1-\pi(x)) & ,f(x)>1\end{cases}\end{aligned}$$

We can minimize this function only when $-1\leq f(x)\leq 1$ thus we have

$$\mathbb{E}[\ell(yf(x))|x]=(1-f(x))\pi(x)+(1+f(x))(1-\pi(x))\quad -1\leq f(x)\leq 1$$

then we have the relation,

$$f^*=\begin{cases}1 & ,\frac{1}{2}<\pi(x)<1\\-1 & ,0<\pi(x)<\frac{1}{2}\\0 & ,\pi(x)=\frac{1}{2}\end{cases}$$

which we write as

$$f^*=\operatorname{sign}(\pi(x)-\frac{1}{2})$$

3 Logistic Regression

3.1 Equivalence of ERM and probabilistic approaches

In lecture we discussed two different ways to end up with logistic regression.

ERM approach: Consider the classification setting with input space $\mathcal{X} = \mathbf{R}^d$, outcome space $\mathcal{Y}_\pm = \{-1, 1\}$, and action space $\mathcal{A}_\mathbf{R} = \mathbf{R}$, with the hypothesis space of linear score functions $\mathcal{F}_{\text{score}} = \left\{x \mapsto x^T w \mid w \in \mathbf{R}^d\right\}$. Consider the margin-based loss function $\ell_{\text{logistic}}(m) = \log\left(1 + e^{-m}\right)$ and the training data $\mathcal{D} = ((x_1, y_1), \ldots, (x_n, y_n))$. Then the empirical risk objective function for hypothesis space $\mathcal{F}_{\text{score}}$ and the logistic loss over $\mathcal{D}$ is given by

$$\begin{aligned}
\hat{R}_n(w) &= \frac{1}{n}\sum_{i=1}^{n} \ell_{\text{logistic}}(y_i w^T x_i) \\
&= \frac{1}{n}\sum_{i=1}^{n} \log\left(1 + \exp\left(-y_i w^T x_i\right)\right).
\end{aligned}$$

Bernoulli regression with logistic transfer function: Consider the conditional probability modeling setting with input space $\mathcal{X} = \mathbf{R}^d$, outcome space $\mathcal{Y}_{0/1} = \{0, 1\}$, and action space $\mathcal{A}_{[0,1]} = [0, 1]$, where an action corresponds to the predicted probability that an outcome is 1. Define the **standard logistic function** as $\phi(\eta) = 1/\left(1 + e^{-\eta}\right)$ and the hypothesis space $\mathcal{F}_{\text{prob}} = \left\{x \mapsto \phi(w^T x) \mid w \in \mathbf{R}^d\right\}$. Suppose for every y_i in the dataset $\mathcal{D}$ above, we define $y_i' = \begin{cases} 1 & y_i = 1 \\ 0 & y_i = -1 \end{cases}$, and let $\mathcal{D}'$ be the resulting collection of (x_i, y_i') pairs. Then the negative log-likelihood (NLL) objective function for $\mathcal{F}_{\text{prob}}$ and $\mathcal{D}'$ is give by

$$\begin{aligned}
\text{NLL}(w) &= -\sum_{i=1}^{n} y_i' \log \phi(w^T x_i) + (1 - y_i') \log\left(1 - \phi(w^T x_i)\right) \\
&= \sum_{i=1}^{n} \left[-y_i' \log \phi(w^T x_i)\right] + (y_i' - 1) \log\left(1 - \phi(w^T x_i)\right)
\end{aligned}$$

If $\hat{w}_{\text{prob}}$ minimizes $\text{NLL}(w)$, then $x \mapsto \phi(x^T \hat{w}_{\text{prob}})$ is a maximum likelihood prediction function over the hypothesis space $\mathcal{F}_{\text{prob}}$ for the dataset $\mathcal{D}'$.

Show that $n\hat{R}_n(w) = \text{NLL}(w)$ for all $w \in \mathbf{R}^d$. And thus the two approaches are equivalent, in that they produce the same prediction functions.

Solution:
We have $\phi(\eta) = 1/\left(1 + e^{-\eta}\right)$, so we can rewrite $\hat{R}_n$ as

$$\hat{R}_n = \frac{1}{n}\sum_{i=1}^{n} \log(\phi(-y_i w^T x_i)^{-1})$$

$$n\hat{R}_n = -\sum_{i=1}^{n} \log(\phi(-y_i w^T x_i))$$

Now we know that y_i can either be 1 or -1. But we want to write it in terms of $y_i^{'}$ which takes on values of $0, 1$.

$$n\hat{R}_n = -\sum_{i=1}^{n}[\underbrace{y_i^{'}\log(\phi(w^T x_i)}_{y_i=1} + \underbrace{(1-y_i^{'})\log(1-\phi(w^T x_i)}_{y_i=-1}]$$

Multiplying the negative through, we have

$$n\hat{R}_n = \sum_{i=1}^{n}[-y_i^{'}\log(\phi(w^T x_i) + (y_i^{'}-1)\log(1-\phi(w^T x_i)] = \text{NLL}$$

3.2 Numerical Overflow and the log-sum-exp trick

Suppose we want to calculate $\log(\exp(\eta))$ for $\eta = 1000.42$. If we compute this literally in Python, we will get an overflow (try it!), since numpy gets infinity for $e^{1000.42}$, and log of infinity is still infinity. On the other hand, we can help out with some math: obviously $\log(\exp(\eta)) = \eta$, and there's no issue.

It turns out, $\log(\exp(\eta))$ and the problem with its calculation is a special case of the LogSumExp function that shows up frequently in machine learning. We define

$$\text{LogSumExp}(x_1, \ldots, x_n) = \log\left(e^{x_1} + \cdots + e^{x_n}\right).$$

Note that this will overflow if any of the x_i's are large (more than 709). To compute this on a computer, we can use the "**log-sum-exp trick**". We let $x^* = \max(x_1, \ldots, x_n)$ and compute LogSumExp as

$$\text{LogSumExp}(x_1, \ldots, x_n) = x^* + \log\left[e^{x_1 - x^*} + \cdots + e^{x_n - x^*}\right].$$

1. Show that the new expression for LogSumExp is valid.
   Solution:
   Let $y = \log(e^{x_1} + \ldots + e^{x_n})$

$$e^y = e^{x_1} + \ldots + e^{x_n}$$

$$e^{-x^*}e^y = (e^{x_1} + \ldots + e^{x_n})e^{-x^*}$$

$$e^{y-x^*} = e^{x_1-x^*} + \ldots + e^{x_n-x^*} = \sum_{i=1}^{n} e^{x_i-x^*}$$

$$y - x^* = \log(\sum_{i=1}^{n} e^{x_i-x^*})$$

$$y = x^* + \log(\sum_{i=1}^{n} e^{x_i-x^*})$$

   For arbitrary x^*. Here, we choose x^* to be $\max(x_1, \ldots, x_n)$ to avoid overflow and underflow.

2. Show that $\exp(x_i - x^*) \in (0, 1]$ for any i, and thus the exp calculations will not overflow.
Solution:
Since we let $x^* = \max(x_1, \ldots, x_n)$, $x_i - x^*$ will either be negative or zero. If $(x_i - x^*)$ is zero, then e^0, we have the upper bound 1. For the negative values, exponential functions must be greater than 0. Therefore, $\exp(x_i - x^*) \in (0, 1]$. This will prevent from calculating large exponential.

3. Above we've only spoken about the exp overflowing. However, the log part can also have problems by becoming negative infinity for arguments very close to 0. Explain why the log term in our expression $\log\left[e^{x_1 - x^*} + \cdots + e^{x_n - x^*}\right]$ will never be "-inf".
Solution:
In $(x_i - x^*)$, there is a $x_t = x^*$, so in the series, $e^{x_1 - x^*} + \ldots + e^{x_n - x^*}$, there is one $e^0 = 1$. Also from the previous problem, since exponential functions are positive, $e^{x_1 - x^*} + \ldots + e^{x_n - x^*} > 1$. This means the log will have arguments greater than 1, thus it will never be "-inf".

4. In the objective functions for logistic regression, there are expressions of the form $\log(1 + e^{-s})$ for some s. Note that a naive implementation gives 0 for $s > 36$ and inf for $s < -709$. Show how to use the numpy function *logaddexp* to correctly compute $\log(1 + e^{-s})$.
Solution:
We can rewrite 1 as e^0 to use *logaddexp*. We just have to compute $\log(e^0 + e^{-s})$.

```
s = 37
p1 = 0
p2 = s
print(np.logaddexp(p1,-p2)) # = 8.533047625744066e-17
print(np.log(1+np.exp(-s))) # = 0.0
```

I tried with $s = 37$ and *logaddexp* works while naive method returns 0.

3.3 Regularized Logistic Regression

For a dataset $\mathcal{D} = ((x_1, y_1), \ldots, (x_n, y_n))$ drawn from $\mathbf{R}^d \times \{-1, 1\}$, the regularized logistic regression objective function can be defined as

$$\begin{aligned} J_{\text{logistic}}(w) &= \hat{R}_n(w) + \lambda \|w\|^2 \\ &= \frac{1}{n} \sum_{i=1}^{n} \log\left(1 + \exp\left(-y_i w^T x_i\right)\right) + \lambda \|w\|^2. \end{aligned}$$

1. Prove that the objective function $J_{\text{logistic}}(w)$ is convex. You may use any facts mentioned in the convex optimization notes.
Solution:
We can rewrite the logistic objective function as

$$J_{\text{logistic}}(w) = \frac{1}{n} \sum_{i=1}^{n} \log(e^0 + e^{-y_i w^T x_i}) + \lambda ||w||^2$$

From the notes, we have that $x_1, x_2, \ldots, x_i \to \log(e^{x_1} + e^{x_2} \ldots + e^{x_i})$ is convex on R^i and $|x|^p$ is convex for $p \geq 1$. Thus an addition of two convex functions is convex, so J_{logistic} is convex.

2. Complete the `f_objective` function in the skeleton code, which computes the objective function for $J_{\text{logistic}}(w)$. Make sure to use the log-sum-exp trick to get accurate calculations and to prevent overflow.
Solution:

```
def f_objective(theta, X, y, l2_param=1):
    num_instances = X.shape[0]
    term2 = l2_param*(np.dot(theta,theta))
    suum = 0
    for i in range(num_instances):
        suum = suum + np.logaddexp(0,-y.values[i]*np.dot(X[i],theta))
    return ((suum/num_instances) + term2)[0]
```

3. Complete the `fit_logistic_regression_function` in the skeleton code using the `minimize` function from `scipy.optimize`. `ridge_regression.py` from Homework 2 gives an example of how to use the `minimize` function. Use this function to train a model on the provided data. Make sure to take the appropriate preprocessing steps, such as standardizing the data and adding a column for the bias term.
Solution:

```
def fit_logistic_reg(X, y, objective_function, l2_param=1):
    row, col = X.shape
    w0 = np.zeros(col)
    opt = minimize(objective_function, w0, args=(X,y,l2_param)).x
    return opt
```

4. Find the ℓ_2 regularization parameter that minimizes the log-likelihood on the validation set. Plot the log-likelihood for different values of the regularization parameter.
Solution:

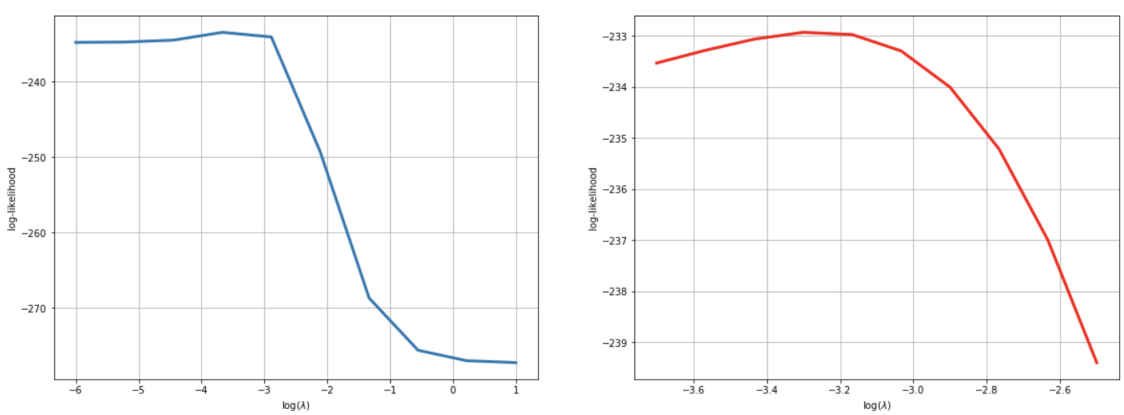


Figure 1: Left: log-likelihood for $\lambda \in [10^{-6}, 10]$ Right: log-likelihood for $\lambda \in [10^{-3.6}, 10^{-2.6}]$

The maximum log-likelihood is $\lambda \approx 10^{-3.2}$.

5. Based on the Bernoulli regression development of logistic regression, it seems reasonable to interpret the prediction $f(x) = \phi(w^T x) = 1/\left(1 + e^{-w^T x}\right)$ as the probability that $y = 1$, for a randomly drawn pair (x, y). Since we only have a finite sample (and we are regularizing, which will bias things a bit) there is a question of how well "calibrated" our predicted probabilities are. Roughly speaking, we say $f(x)$ is well calibrated if we look at all examples (x, y) for which $f(x) \approx 0.7$ and we find that close to 70% of those examples have $y = 1$, as predicted... and then we repeat that for all predicted probabilities in $(0, 1)$. To see how well-calibrated our predicted probabilities are, break the predictions on the validation set into groups based on the predicted probability (you can play with the size of the groups to get a result you think is informative). For each group, examine the percentage of positive labels. You can make a table or graph. Summarize the results. You may get some ideas and references from scikit-learn's discussion.
Solution:

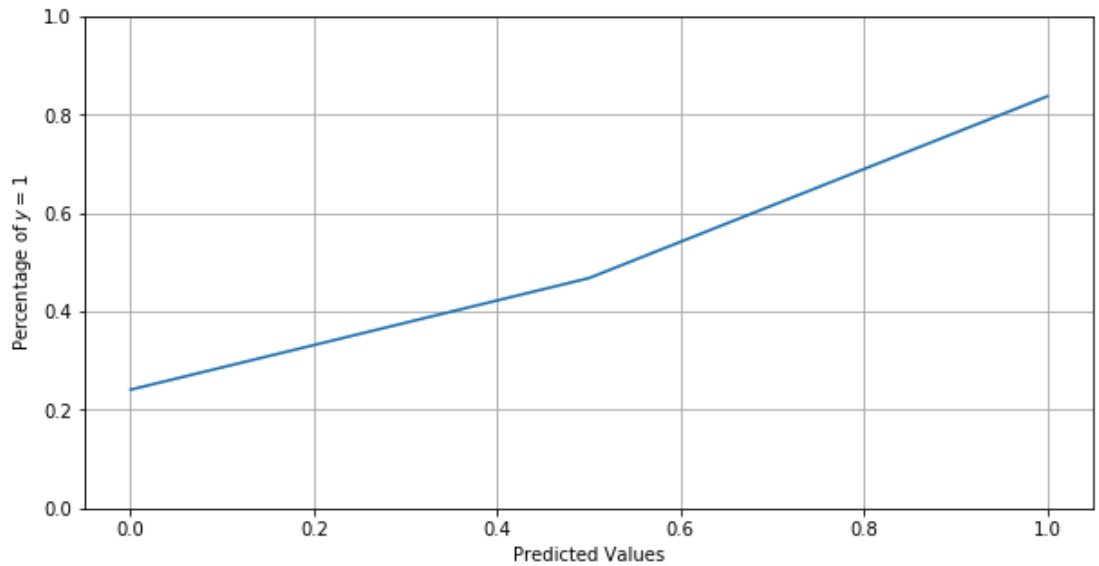


Figure 2: Plot of the prediction values and percentage of $y = 1$ of each calibration group.

For this problem, I broke the group into 5 groups with 0.2 percentage gaps. The plot is almost a straight line which means the prediction function is well-calibrated.

6. [Optional] If you can, create a dataset for which the log-sum-exp trick is actually necessary for your implementation of regularized logistic regression. If you don't think such a dataset exists, explain why. If you like, you may consider the case of SGD optimization. [This problem is intentionally open-ended. You're meant to think, explore, and experiment. Points assigned for interesting insights.]
Solution:
From the above explanation, $\text{LogSumExp}(x_1, \ldots, x_n) = \log(e^{x_1} + \ldots + e^{x_n})$ overflows if any if the $x_i s$ are greater than 36 or less than -709. We use the LogSumExp trick to calculate the objective function. In the objective function, the term that might cause overflow is $-y_i w^T x_i$. Suppose we initialize w to be 1, given a large value of x_i, it is possible that $-y_i w^T x_i <$ -709 or $>$ 36. Thus LogSumExp is necessary when we have large values of x_is.

4 Bayesian Logistic Regression with Gaussian Priors

Let's return to the setup described in Section 3.1 and, in particular, to the Bernoulli regression setting with logistic transfer function. We had the following hypothesis space of conditional probability functions:

$$\mathcal{F}_{\text{prob}} = \left\{ x \mapsto \phi(w^T x) \mid w \in \mathbf{R}^d \right\}.$$

Now let's consider the Bayesian setting, where we induce a prior on $\mathcal{F}_{\text{prob}}$ by taking a prior $p(w)$ on the parameter $w \in \mathbf{R}^d$.

1. For the dataset $\mathcal{D}'$ described in Section 3.1, give an expression for the posterior density $p(w \mid \mathcal{D}')$ in terms of the negative log-likelihood function

$$\text{NLL}_{\mathcal{D}'}(w) \quad = \quad -\sum_{i=1}^{n} y_i' \log \phi(w^T x_i) + (1 - y_i') \log \left(1 - \phi(w^T x_i)\right)$$

and a prior density $p(w)$ (up to a proportionality constant is fine).
Solution:

$$\text{posterior density} = p(w|\mathcal{D}') \propto \text{prior} \times \text{likelihood}$$
$$p(w|\mathcal{D}') \propto p(w) e^{-\text{NLL}}$$

2. Suppose we take a prior on w of the form $w \sim \mathcal{N}(0, \Sigma)$. Find a covariance matrix Σ such that MAP estimate for w after observing data $\mathcal{D}'$ is the same as the minimizer of the regularized logistic regression function defined in Section 3.3 (and prove it). [Hint: Consider minimizing the negative log posterior of w. Also, remember you can drop any terms from the objective function that don't depend on w. Also, you may freely use results of previous problems.]
Solution:
From the previous problem, we have

$$p(w|\mathcal{D}') \propto p(w) e^{-\text{NLL}}$$

Plugging in the prior,

$$p(w|\mathcal{D}') \propto (\frac{1}{\sqrt{(2\pi)^k|\Sigma|}} e^{-\frac{1}{2}w^T\Sigma^{-1}w})e^{-\text{NLL}}$$

Taking the negative log of posterior density,

$$-\log(p(w|\mathcal{D}')) \propto -\log((\frac{1}{\sqrt{(2\pi)^k|\Sigma|}} e^{-\frac{1}{2}w^T\Sigma^{-1}w})e^{-\text{NLL}})$$

Using the relation $n\hat{R} = \text{NLL}$

$$-\log(p(w|\mathcal{D}')) \propto -[\log(\frac{1}{\sqrt{(2\pi)^k|\Sigma|}}) + (-\frac{1}{2}w^T\Sigma^{-1}w - n\hat{R})]$$

$$-\log(p(w|\mathcal{D}')) \propto -[(-\frac{1}{2}w^T\Sigma^{-1}w - n\hat{R})] = \frac{1}{2}w^T\Sigma^{-1}w + n\hat{R}$$

(Excluding constants)
Our objective function from 3.3 is

$$J_{\text{logistic}}(w) \quad = \quad \hat{R}_n(w) + \lambda\|w\|^2$$

We are trying to minimize both objective function and the negative log posterior of w.

$$\min(\frac{1}{2}w^T\Sigma^{-1}w + n\hat{R}) = \min(n\hat{R} + \lambda n||w||^2)$$

Comparing it with the negative log of posterior density, we can see that

$$n\lambda||w||^2 = \frac{w^T\Sigma^{-1}w}{2}$$

therefore, in order satisfy the inequality, $\Sigma = \frac{1}{2n\lambda}I$ where I is and identity matrix.

3. In the Bayesian approach, the prior should reflect your beliefs about the parameters before seeing the data and, in particular, should be independent on the eventual size of your dataset. Following this, you choose a prior distribution $w \sim \mathcal{N}(0, I)$. For a dataset $\mathcal{D}$ of size n, how should you choose λ in our regularized logistic regression objective function so that the minimizer is equal to the mode of the posterior distribution of w (i.e. is equal to the MAP estimator).
Solution:
From the previous problem, we have

$$n\lambda||w||^2 = \frac{w^T(I)^{-1}w}{2}$$

Therefore, $\lambda = \frac{1}{2n}$, so that regularized logistic regression objective function is equal to MAP estimator.

5 Bayesian Linear Regression - Implementation

In this problem, we will implement Bayesian Gaussian linear regression, essentially reproducing the example from lecture, which in turn is based on the example in Figure 3.7 of Bishop's *Pattern Recognition and Machine Learning* (page 155). We've provided plotting functionality in "support_code.py". Your task is to complete "problem.py". The implementation uses np.matrix objects, and you are welcome to use[4] the np.matrix.getI method.

1. Implement likelihood_func.
   Solution:

```
def likelihood_func(w, X, y_train, likelihood_var):
    row,col = X.shape
    likelihood = 1.0
    for i in range(row):
        prob = np.exp(-((y_train[i]-np.dot(X[i],w))**2)/(2*likelihood_var)) *
    1/np.sqrt(2*np.pi*likelihood_var)
        likelihood = likelihood * prob
    return likelihood
```

2. Implement get_posterior_params.
   Solution:

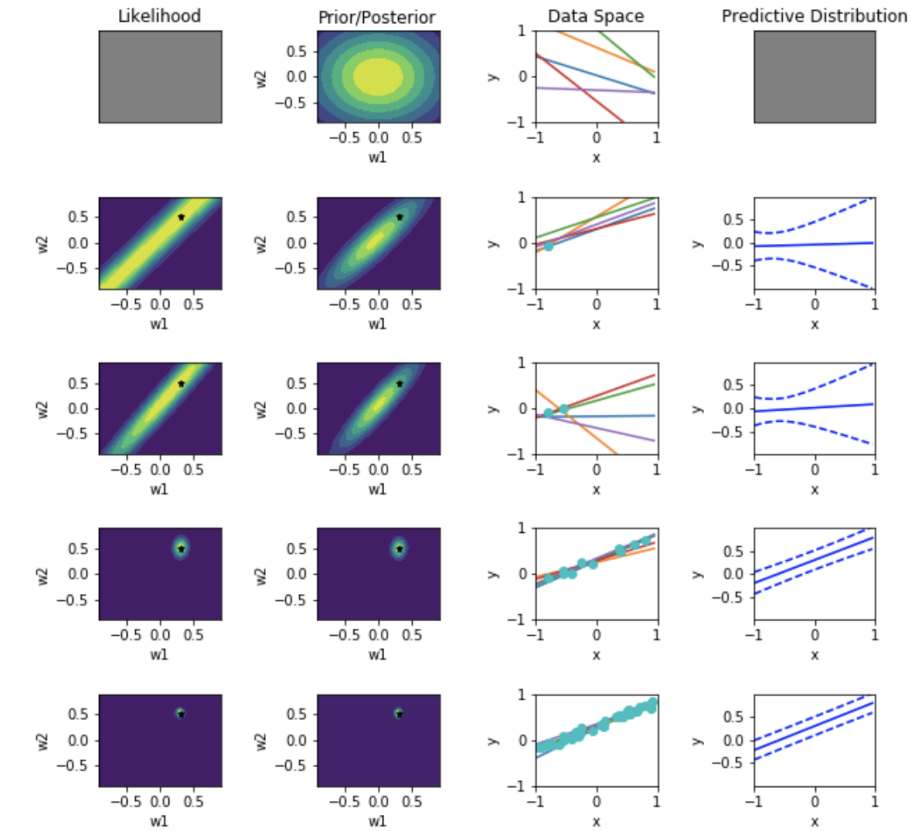
```
def get_posterior_params(X, y_train, prior, likelihood_var = 0.2**2):
    mu1 = np.dot(X.T,X) + likelihood_var*prior['var'].getI()
    mu2 = np.dot(X.T,y_train)
    post_mean = mu1.getI()*mu2
    post_var = ((1./likelihood_var)*np.dot(X.T,X) + prior['var'].getI()).getI()
    return post_mean, post_var
```

3. Implement get_predictive_params.
   Solution:

```
def get_predictive_params(X_new, post_mean, post_var, likelihood_var = 0.2**2):
    pred_mean = np.dot(post_mean.T,X_new)
    pred_var = np.dot(X_new.T,np.dot(post_var,X_new)) + likelihood_var
    return pred_mean, pred_var
```

4. Run "python problem.py" from inside the Bayesian Regression directory to do the regression and generate the plots. This runs through the regression with three different settings for the prior covariance. You may want to change the default behavior in support_code.make_plots from plt.show, to saving the plots for inclusion in your homework submission.
   Solution:

[4] However, in practice we are usually interested in computing the product of a matrix inverse and a vector, i.e. $X^{-1}b$. In this case, it's usually faster and more accurate to use a library's algorithms for solving a system of linear equations. Note that $y = X^{-1}b$ is just the solution to the linear system $Xy = b$. See for example John Cook's blog post for discussion.



5. Comment on your results. In particular, discuss how each of the following change with sample size and with the strength of the prior: (i) the likelihood function, (ii) the posterior distribution, and (iii) the posterior predictive distribution.
   Solution:
   The difference in between the three plots are the posterior and prior distributions. Models with very small variance for prior constraints the predictive distribution. Whereas models with higher variance allows more flexibility.
   In each row of the plots, the sample size given to the model in increases. As the sample size increases, (i) the parameters for likelihood function w_2, w_1 is starting to localized. (ii) the posterior distribution are also localizing in the true parameters. (iii) predictive distribution fits better as sample sizes increases because the parameters are localized near the true parameters.

   Note that for the last plot (with the smallest variance), the posterior contour plot does not localize near the true parameters because of the constraints.

6. Our work above was very much "full Bayes", in that rather than coming up with a single prediction function, we have a whole distribution over posterior prediction functions. However, sometimes we want a single prediction function, and a common approach is to use the MAP

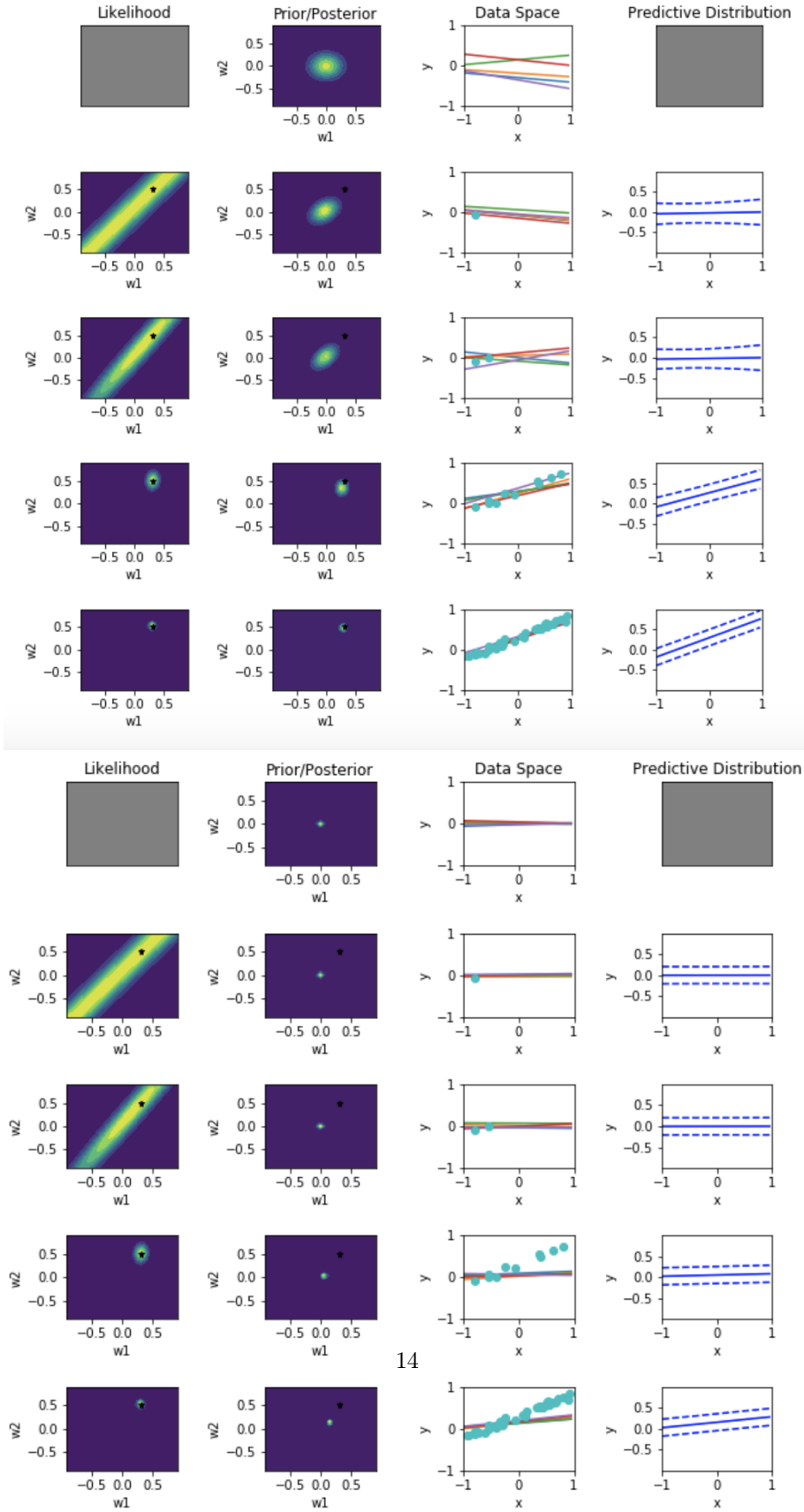


Figure 3: Plots of Likelihood, Prior/Posterior, Data Space, Predictive Distribution for $\Sigma_0 = [\frac{1}{2}I, \frac{1}{2^5}I, \frac{1}{2^{10}}I]$ as the model sees more observations

estimate – that is, choose the prediction function that has the highest posterior likelihood. As we discussed in class, for this setting, we can get the MAP estimate using ridge regression. Use ridge regression to get the MAP prediction function corresponding to the first prior covariance ($\Sigma = \frac{1}{2}I$, per the support code). What value did you use for the regularization coefficient? Why?

Solution:

With the prior variance, $\Sigma_0 = \frac{\sigma^2}{\lambda}I$, where σ^2 is the likelihood variance and from the previous problem, we set $\lambda = \frac{1}{2}$. Using the above parameters to fit sklearn's regression, the final coefficients are: $[0.30052135, 0.52406189]$

```
In [21]: alpha = likelihood_var/(1/2) # Change to the correct value
         ridge = Ridge(alpha=alpha,
                       fit_intercept=False,
                       solver='cholesky')

         ridge.fit(xtrain, ytrain)

Out[21]: Ridge(alpha=0.08000000000000002, copy_X=True, fit_intercept=False,
            max_iter=None, normalize=False, random_state=None, solver='cholesky',
            tol=0.001)
```

If alpha is set correctly, ridge.coef_ will equal the prior mean/MAP estimate returned by the next two cells.

```
In [10]: ridge.coef_

Out[10]: array([[0.30052135, 0.52406189]])
```

```
In [32]: prior = {"mean":np.matrix([[0], [0]]),
                  "var":matlib.eye(2) * sigmas[0]}
         try:
             post = get_posterior_params(xtrain, ytrain, prior,
                                         likelihood_var = 0.2**2)
             post[0].ravel()
         except NameError:
             print('If not yet implemented, implement functions in problem.py.')
             print('If you have implemented, remove this try/except.')
```

```
In [12]: post[0].ravel()

Out[12]: matrix([[0.30052135, 0.52406189]])
```

6 [Optional] Coin Flipping: Maximum Likelihood

1. [Optional] Suppose we flip a coin and get the following sequence of heads and tails:

$$\mathcal{D} = (H, H, T)$$

Give an expression for the probability of observing $\mathcal{D}$ given that the probability of heads is θ. That is, give an expression for $p(\mathcal{D} \mid \theta)$. This is called the **likelihood of θ for the data $\mathcal{D}$**.

Solution:

$$p(\mathcal{D}|\theta) = \theta^2(1-\theta)$$

2. [Optional] How many different sequences of 3 coin tosses have 2 heads and 1 tail? If we toss the coin 3 times, what is the probability of 2 heads and 1 tail? (Answer should be in terms of θ.)
Solution:
There can be 3 sequences that have 2 heads and 1 tail. {H,H,T},{H,T,H},{T,H,H}. (I just looked at the positions that T can be in)
The probability of getting 2 heads and 1 tail is:

$$p(H = 2, T = 1|\theta) = \theta^2(1-\theta) + \theta(1-\theta)\theta + (1-\theta)\theta^2$$

where θ is the probability of getting H.

3. [Optional] More generally, give an expression for the likelihood $p(\mathcal{D} \mid \theta)$ for a particular sequence of flips $\mathcal{D}$ that has n_h heads and n_t tails. Make sure you have expressions that make sense even for $\theta = 0$ and $n_h = 0$, and other boundary cases. You may use the convention that $0^0 = 1$, or you can break your expression into cases if needed.
Solution:

$$p(\mathcal{D}|\theta) = \prod_{i=1}^{n} (\theta)^{x_i}(1-\theta)^{1-x_i}$$

where x_i is an indicator function that returns 1 if head is observed and 0 if tails is observed. We can alternatively write the above expression in terms of number of heads observed and number of tails observed as

$$p(\mathcal{D}|\theta) = (\theta)^{n_h}(1-\theta)^{n_t}$$

4. [Optional] Show that the maximum likelihood estimate of θ given we observed a sequence with n_h heads and n_t tails is

$$\hat{\theta}_{\text{MLE}} = \frac{n_h}{n_h + n_t}.$$

You may assume that $n_h + n_t \geq 1$. (Hint: Maximizing the log-likelihood is equivalent and is often easier.)
Solution:
Let's first take the log of the likelihood function,

$$\log(p(\mathcal{D}|\theta)) = \log((\theta)^{n_h}(1-\theta)^{n_t})$$

$$= n_h \log(\theta) + n_t \log(1-\theta)$$

Taking the derivative and setting it to zero to find the maximum,

$$\frac{\partial}{\partial\theta} \log(p(\mathcal{D}|\theta)) = \frac{n_h}{\theta} - \frac{n_t}{1-\theta} = 0$$

Solving for θ we get

$$\hat{\theta}_{\text{MLE}} = \frac{n_h}{n_t + n_h}$$

7 [Optional] Coin Flipping: Bayesian Approach with Beta Prior

We'll now take a Bayesian approach to the coin flipping problem, in which we treat θ as a random variable sampled from some prior distribution $p(\theta)$. We'll represent the ith coin flip by a random variable $X_i \in \{0, 1\}$, where $X_i = 1$ if the ith flip is heads. We assume that the X_i's are conditionally independent given θ. This means that the joint distribution of the coin flips and θ factorizes as follows:

$$\begin{aligned} p(x_1, \ldots, x_n, \theta) &= p(\theta) p(x_1, \ldots, x_n \mid \theta) \text{ (always true)} \\ &= p(\theta) \prod_{i=1}^{n} p(x_i \mid \theta) \text{ (by conditional independence).} \end{aligned}$$

1. [Optional] Suppose that our prior distribution on θ is Beta(h, t), for some $h, t > 0$. That is, $p(\theta) \propto \theta^{h-1} (1-\theta)^{t-1}$. Suppose that our sequence of flips $\mathcal{D}$ has n_h heads and n_t tails. Show that the posterior distribution for θ is Beta$(h + n_h, t + n_t)$. That is, show that

$$p(\theta \mid \mathcal{D}) \propto \theta^{h-1+n_h} (1-\theta)^{t-1+n_t}.$$

We say that the Beta distribution is **conjugate** to the Bernoulli distribution since the prior and the posterior are both in the same family of distributions (i.e. both Beta distributions).
Solution:
We have

$$p(\theta) \propto \theta^{h-1} (1-\theta)^{t-1}$$

as prior and with our likelihood function from problem 6.3,

$$p(\theta \mid \mathcal{D}) \propto \underbrace{\theta^{h-1}(1-\theta)^{t-1}}_{\text{prior}} \underbrace{(\theta^{n_h})(1-\theta)^{n_t}}_{\text{likelihood}}$$

$$p(\theta \mid \mathcal{D}) \propto \theta^{h-1+n_h} (1-\theta)^{t-1+n_t}$$

$$p(\theta \mid \mathcal{D}) \sim \text{Beta}(h + n_h, t + n_t)$$

2. [Optional] Give expressions for the MLE, the MAP, and the posterior mean estimates of θ. [Hint: You may use the fact that a Beta(h, t) distribution has mean $h/(h+t)$ and has mode $(h-1)/(h+t-2)$ for $h, t > 1$. For the Bayesian solutions, you should note that as $h + t$ gets very large, and assuming we keep the ratio $h/(h+t)$ fixed, the posterior mean and MAP approach the prior mean $h/(h+t)$, while for fixed h and t, the posterior mean approaches the MLE as the sample size $n = n_h + n_t \to \infty$.
Solution:
Let's first find the maximum likelihood estimator. The likelihood function (from problem 6.3),

$$p(\mathcal{D}|\theta) = \theta^{n_h}(1-\theta)^{n_t}$$

Taking the log and derivative to find the maximum θ,

$$\log(p(\mathcal{D}|\theta)) = \log(\theta) n_h + \log((1-\theta)) n_t$$

$$\frac{\partial}{\partial\theta}\log(p(\mathcal{D}|\theta)) = \frac{1}{\theta}n_h - \frac{1}{1-\theta}n_t = 0$$

Solving for θ,

$$\hat{\theta}_{\text{MLE}} = \frac{n_h}{n_h + n_t}$$

Let's solve for the MAP estimator, which is the θ that maximizes the posterior distribution. From the previous problem, we have the following posterior distribution,

$$p(\theta \mid \mathcal{D}) \propto \theta^{h-1+n_h}\,(1-\theta)^{t-1+n_t}$$

Taking the log and finding its maximizer,

$$\log(p(\theta \mid \mathcal{D})) \propto (h + n_h - 1)\log(\theta) + (t + n_t - 1)(1-\theta)$$

$$\frac{\partial}{\partial\theta}\log(p(\theta \mid \mathcal{D})) \propto \frac{(h + n_h - 1)}{\theta} - \frac{(t + n_t - 1)}{1-\theta} = 0$$

$$(1-\theta)(h + n_h - 1) = \theta(t + n_t - 1)$$

$$\hat{\theta}_{\text{MAP}} = \frac{h + n_h - 1}{h + t + n_h + n_t - 2}$$

Lastly, the posterior mean is the mean of $\text{Beta}(h + n_h, t + n_t)$

$$\hat{\theta}_{\text{Posterior Mean}} = \frac{h + n_h}{h + n_h + t + n_t}$$

3. [Optional] What happens to $\hat{\theta}_{\text{MLE}}$, $\hat{\theta}_{\text{MAP}}$, and $\hat{\theta}_{\text{POSTERIOR MEAN}}$ as the number of coin flips $n = n_h + n_t$ approaches infinity?
Solution:
As the number of total flips approaches infinity, the maximum likelihood estimator limits to θ, by the law of large numbers.
For MAP and Posterior Mean, as $n \to \infty$, both estimators approximates to the maximum likelihood estimator, thus they both are a consistent estimator for θ. (It converges in probability to θ.)

4. [Optional] The MAP and posterior mean estimators of θ were derived from a Bayesian perspective. Let's now evaluate them from a frequentist perspective. Suppose θ is fixed and unknown. Which of the MLE, MAP, and posterior mean estimators give **unbiased** estimates of θ, if any? [Hint: The answer may depend on the parameters h and t of the prior. Also, let's consider the total number of flips $n = n_h + n_t$ to be given (not random), while n_h and n_t are random, with $n_h = n - n_t$.]
Solution:
The definition of bias is

$$\mathbb{E}[\hat{\theta}_{\text{estimator}}] - \theta = 0$$

If the estimator is unbiased.
Let's start with MLE. Rewriting MLE in terms of indicator function if the observation is H,

$$\mathbb{E}[\hat{\theta}_{\text{MLE}}] = \mathbb{E}[\frac{\sum_{i=1}^{n} x_i}{n}] = \frac{1}{n}\,\mathbb{E}[\sum_{i=1}^{n} x_i] = \theta$$

MLE is an unbiased estimator of θ.
Now for MAP,

$$\mathbb{E}[\hat{\theta}_{\text{MAP}}] = \mathbb{E}[\frac{h + \sum_{i=1}^{n} x_i - 1}{h + t + n - 2}] = \frac{1}{h + t + n - 2}\,\mathbb{E}[h + \sum_{i=1}^{n} x_i - 1] = \frac{h - \theta n - 1}{h + t + n - 2} \neq \theta$$

Therefore, MAP is a biased estimator!
Lastly, for Posterior Mean estimate,

$$\mathbb{E}[\hat{\theta}_{\text{Posterior Mean}}] = \mathbb{E}[\frac{h + \sum_{i=1}^{n} x_i}{h + t + n}] = \frac{1}{h + t + n}\,\mathbb{E}[h + \sum_{i=1}^{n} x_i] = \frac{h + n\theta}{h + t + n} \neq \theta$$

Thus, posterior mean is also a biased estimator!

5. [Optional] Suppose somebody gives you a coin and asks you to give an estimate of the probability of heads, but you can only toss the coin 3 times. You have no particular reason to believe this is an unfair coin. Would you prefer the MLE or the posterior mean as a point estimate of θ? If the posterior mean, what would you use for your prior?
Solution:
Essentially, the difference between MLE and posterior mean is that one assumes a uniform prior and the other allow some arbitrary prior.
Suppose I tossed a coin 3 times, and the result is {H,H,H}. Then, $\hat{\theta}_{\text{MLE}} = 1$, however, if we use Beta as a prior, then, from the previous problem, we know that $\hat{\theta}_{\text{Posterior Mean}} < 1$. I would prefer the posterior mean because for small trials, it acts as a constraint, and as the number of trials increases, it approaches the true value.

8 [Optional] Hierarchical Bayes for Click-Through Rate Estimation

In mobile advertising, ads are often displayed inside apps on a phone or tablet device. When an ad is displayed, this is called an "impression." If the user clicks on the ad, that is called a "click." The probability that an impression leads to a click is called the "click-through rate" (CTR).

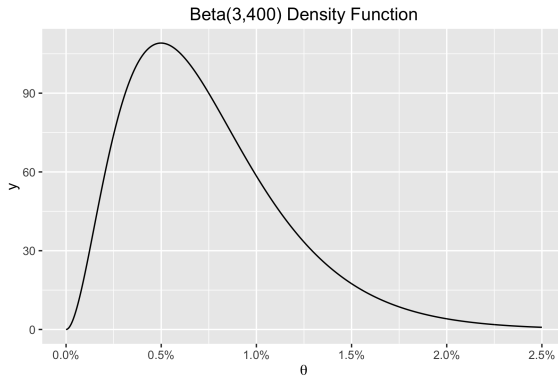
Suppose we have $d = 1000$ apps. For various reasons[5], each app tends to have a different overall CTR. For the purposes of designing an ad campaign, we want estimates of all the app-level CTRs, which we'll denote by $\theta_1, \ldots, \theta_{1000}$. Of course, the particular user seeing the impression and the particular ad that is shown have an effect on the CTR, but we'll ignore these issues for now. [Because so many clicks on mobile ads are accidental, it turns out that the overall app-level CTR often dominates the effect of the particular user and the specific ad.]

If we have enough impressions for a particular app, then the empirical fraction of clicks will give a good estimate for the actual CTR. However, if we have relatively few impressions, we'll have some problems using the empirical fraction. Typical CTRs are less than 1%, so it takes a fairly large number of observations to get a good estimate of CTR. For example, even with 100 impressions, the only possible CTR estimates given by the MLE would be $0\%, 1\%, 2\%, \ldots, 100\%$. The 0% estimate

[5]The primary reason is that different apps place the ads differently, making it more or less difficult to avoid clicking the ad.

is almost certainly much too low, and anything 2% or higher is almost certainly much too high. Our goal is to come up with reasonable point estimates for $\theta_1, \ldots, \theta_{1000}$, even when we have very few observations for some apps.

If we wanted to apply the Bayesian approach worked out in the previous problem, we could come up with a prior that seemed reasonable. For example, we could use the following Beta$(3, 400)$ as a prior distribution on each θ_i:



In this basic Bayesian approach, the parameters 3 and 400 would be chosen by the data scientist based on prior experience, or "best guess", but without looking at the new data. Another approach would be to use the data to help you choose the parameters a and b in Beta(a, b). This would **not** be a Bayesian approach, though it is frequently used in practice. One method in this direction is called **empirical Bayes**. Empirical Bayes can be considered a frequentist approach, in which we estimate a and b from the data $\mathcal{D}$ using some estimation technique, such as maximum likelihood. The proper Bayesian approach to this type of thing is called **hierarchical Bayes**, in which we put another prior distribution on a and b. We'll investigate each of these approaches below.

Mathematical Description

We'll now give a mathematical description of our model, assuming the prior parameters a and b are directly chosen by the data scientist. Let $n_1, \ldots, n_d$ be the number of impressions we observe for each of the d apps. In this problem, we will not consider these to be random numbers. For the ith app, let $c_i^1, \ldots, c_i^{n_i} \in \{0, 1\}$ be indicator variables determining whether or not each impression was clicked. That is, $c_i^j = 1(j\text{th impression on } i\text{th app was clicked})$. We can summarize the data on the ith app by $\mathcal{D}_i = (x_i, n_i)$, where $x_i = \sum_{j=1}^{n_i} c_i^j$ is the total number of impressions that were clicked for app i. Let $\theta = (\theta_1, \ldots, \theta_d)$, where θ_i is the CTR for app i.

In our Bayesian approach, we act as though the data were generated as follows:

1. Sample $\theta_1, \ldots, \theta_d$ i.i.d. from Beta(a, b).
2. For each app i, sample $c_i^1, \ldots, c_i^{n_i}$ i.i.d. from Bernoulli(θ_i).

8.1 [Optional] Empirical Bayes for a single app

We start by working out some details for Bayesian inference for a single app. That is, suppose we only have the data $\mathcal{D}_i$ from app i, and nothing else. Mathematically, this is exactly the same setting as the coin tossing setting above, but here we push it further.

1. Give an expression for $p(\mathcal{D}_i \mid \theta_i)$, the likelihood of $\mathcal{D}_i$ given the probability of click θ_i, in terms of θ_i, x_i and n_i.
Solution:

$$p(\mathcal{D}_i|\theta_i) = \prod_{j=1}^{n_i} \theta^{x_j}(1-\theta)^{1-x_j}$$

$$p(\mathcal{D}_i|\theta_i) = \theta^{x_i}(1-\theta)^{n_i-x_i}$$

2. We will take our prior distribution on θ_i to be $\text{Beta}(a, b)$. The corresponding probability density function is given by

$$p(\theta_i) = \text{Beta}(\theta_i; a, b) = \frac{1}{B(a,b)}\theta_i^{a-1}\left(1-\theta_i\right)^{b-1},$$

where $B(a, b)$ is called the Beta function. Explain (without calculation) why we must have

$$\int \theta_i^{a-1}\left(1-\theta_i\right)^{b-1}\, d\theta_i = B(a,b).$$

Solution:
This Beta function act as a normalizing constant to ensure the probability density function of θ_i is within the probability range ($[0, 1]$).

3. Give an expression for the posterior distribution $p(\theta_i \mid \mathcal{D}_i)$. In this case, include the constant of proportionality. In other words, do not use the "is proportional to" sign $\propto$ in your final expression. You **may** reference the Beta function defined above. [Hint: This problem is essentially a repetition of an earlier problem.]
Solution:

$$p(\theta_i|\mathcal{D}_i) = \underbrace{p(\theta_i)}_{\text{Prior}} \underbrace{\prod_{j=1}^{n_i} p(x_i|\theta_i)}_{\text{Likelihood}}$$

$$p(\theta_i|\mathcal{D}_i) = \frac{1}{\text{B}(a,b)}\theta_i^{a-1}(1-\theta_i)^{b-1}\theta^{x_j}(1-\theta)^{n_i-x_j}$$

$$p(\theta_i|\mathcal{D}_i) = \frac{1}{\text{B}(a,b)}\theta_i^{a-1+x_j}(1-\theta_i)^{b-1+n_i-x_j}$$

4. Give a closed form expression for $p(\mathcal{D}_i)$, the marginal likelihood of $\mathcal{D}_i$, in terms of the a, b, x_i, and n_i. You may use the normalization function $B(\cdot, \cdot)$ for convenience, but you should not have any integrals in your solution. (Hint: $p(\mathcal{D}_i) = \int p\left(\mathcal{D}_i \mid \theta_i\right) p(\theta_i)\, d\theta_i$, and the answer will be a ratio of two beta function evaluations.)
Solution:
Plugging in the likelihood and prior into the integral,

$$p(\mathcal{D}_i) = \int p\left(\mathcal{D}_i \mid \theta_i\right) p(\theta_i)\, d\theta_i$$

$$p(\mathcal{D}_i) = \int \theta^{x_i}(1-\theta)^{n_i - x_j} \frac{1}{B(a,b)} \theta_i^{a-1} (1-\theta_i)^{b-1} \, d\theta_i$$

Combining like-terms,

$$p(\mathcal{D}_i) = \int \theta^{x_i + a - 1}(1-\theta)^{n_i - \sum_{j=1}^{n_i} x_j + b - 1} \frac{1}{B(a,b)} d\theta_i$$

$$p(\mathcal{D}_i) = \frac{1}{B(a,b)} \int \theta^{x_i + a - 1}(1-\theta)^{n_i - x_i + b - 1} d\theta_i$$

$$p(\mathcal{D}_i) = \frac{B(a + x_i, b + n_i - x_i)}{B(a,b)}$$

5. The maximum likelihood estimate for θ_i is x_i/n_i. Let $p_{\text{MLE}}(\mathcal{D}_i)$ be the marginal likelihood of $\mathcal{D}_i$ when we use a prior on θ_i that puts all of its probability mass at x_i/n_i. Note that

$$\begin{aligned} p_{\text{MLE}}(\mathcal{D}_i) &= p\left(\mathcal{D}_i \mid \theta_i = \frac{x_i}{n_i}\right) p\left(\theta_i = \frac{x_i}{n_i}\right) \\ &= p\left(\mathcal{D}_i \mid \theta_i = \frac{x_i}{n_i}\right). \end{aligned}$$

Explain why, or prove, that $p_{\text{MLE}}(\mathcal{D}_i)$ is larger than $p(\mathcal{D}_i)$ for any other prior we might put on θ_i. If it's too hard to reason about all possible priors, it's fine to just consider all Beta priors. [Hint: This does not require much or any calculation. It may help to think about the integral $p(\mathcal{D}_i) = \int p(\mathcal{D}_i \mid \theta_i)\, p(\theta_i)\, d\theta_i$ as a weighted average of $p(\mathcal{D}_i \mid \theta_i)$ for different values of θ_i, where the weights are $p(\theta_i)$.]

Solution:

The maximum likelihood can be interpret as a posterior distribution given that the prior a uniform distribution. So, the second term evaluates to 1. Now, if we use some other priors, this term will be less than 1, since it is a probability. Therefore the posterior distribution with a prior will be less than the maximum likelihood.

6. One approach to getting an **empirical Bayes** estimate of the parameters a and b is to use maximum likelihood. Such an empirical Bayes estimate is often called an **ML-2** estimate, since it's maximum likelihood, but at a higher level in the Bayesian hierarchy. To emphasize the dependence of the likelihood of $\mathcal{D}_i$ on the parameters a and b, we'll now write it as $p(\mathcal{D}_i \mid a, b)$[6]. The empirical Bayes estimates for a and b are given by

$$(\hat{a}, \hat{b}) = \underset{(a,b) \in (0,\infty) \times (0,\infty)}{\arg\max} \; p(\mathcal{D}_i \mid a, b).$$

To make things concrete, suppose we observed $x_i = 3$ clicks out of $n_i = 500$ impressions. A plot of $p(\mathcal{D}_i \mid a, b)$ as a function of a and b is given in Figure 4. It appears from this plot that the likelihood will keep increasing as a and b increase, at least if a and b maintain a particular ratio. Indeed, this likelihood function never attains its maximum, so we cannot use ML-2 here. Explain what's happening to the prior as we continue to increase the likelihood. [Hint:

[6] Note that this is a slight (though common) abuse of notation, because a and b are not random variables in this setting. It might be more appropriate to write this as $p(\mathcal{D}_i; a, b)$ or $p_{a,b}(\mathcal{D}_i)$. But this isn't very common.

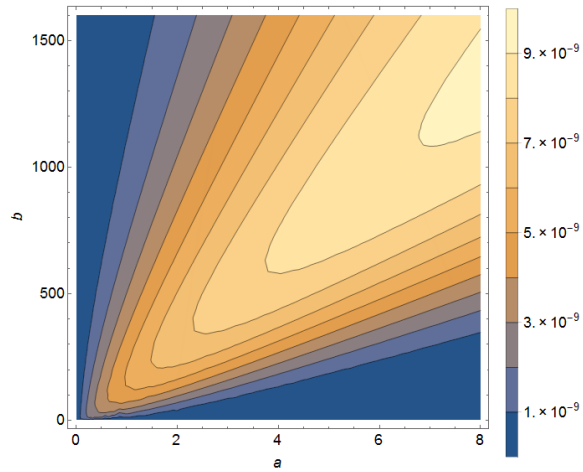


Figure 4: A plot of $p(\mathcal{D}_i \mid a, b)$ as a function of a and b.

It is a property of the Beta distribution (not difficult to see), that for any $\theta \in (0, 1)$, there is a Beta distribution with expected value θ and variance less than ε, for any $\varepsilon > 0$. What's going in here is similar to what happens when you attempt to fit a gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ to a single data point using maximum likelihood.]

Solution:

From the problem, we know that the prior distribution on θ_i is Beta(a, b), which means θ_i depends on parameters a and b. From the hint, for $\theta \in (0, 1)$, there is a Beta distribution with expected value θ and variance less than ϵ for any $\epsilon > 0$. We can pick arbitrary large parameters to reduce the variance on θ and still be supported. By reducing the variance, we will be sampling more around the mean.

8.2 [Optional] Empirical Bayes Using All App Data

In the previous section, we considered working with data from a single app. With a fixed prior, such as Beta(3,400), our Bayesian estimates for θ_i seem more reasonable to me[7] than the MLE when our sample size n_i is small. The fact that these estimates seem reasonable is an immediate consequence of the fact that I chose the prior to give high probability to estimates that seem reasonable to me, before ever seeing the data. Our earlier attempt to use empirical Bayes (ML-2) to choose the prior in a data-driven way was not successful. With only a single app, we were essentially overfitting the prior to the data we have. In this section, we'll consider using the data from all the apps, in which case empirical Bayes makes more sense.

1. Let $\mathcal{D} = (\mathcal{D}_1, \ldots, \mathcal{D}_d)$ be the data from all the apps. Give an expression for $p(\mathcal{D} \mid a, b)$, the **marginal likelihood** of $\mathcal{D}$. Expression should be in terms of a, b, x_i, n_i for $i = 1, \ldots, d$. Assume data from different apps are independent. (Hint: This problem should be easy, based on a problem from the previous section.)

[7] I say "to me", since I am the one who chose the prior. You may have an entirely different prior, and think that my estimates are terrible.

Solution:

$$p(D|a,b) = \prod_{i=1}^{d} p(D_i|a,b) = \prod_{i=1}^{d} \frac{B(a+x_i, b+n_i-x_i)}{B(a,b)}$$

2. Explain why $p(\theta_i \mid \mathcal{D}) = p(\theta_i \mid \mathcal{D}_i)$, according to our model. In other words, once we choose values for parameters a and b, information about one app does not give any information about other apps.
Solution:
Since for each $\mathcal{D}_i$, it depends on θ_i, but each θ_i is identically and independently sampled from Beta(3,400). Therefore, no matter what $\mathcal{D}_i$ we used, θ_i is still iid.

3. Suppose we have data from 6 apps. 3 of the apps have a fair number of impressions, and 3 have relatively few. Suppose we observe the following:

	Num Clicks	Num Impressions
App 1	50	10000
App 2	160	20000
App 3	180	60000
App 4	0	100
App 5	0	5
App 6	1	2

Compute the empirical Bayes estimates for a and b. (Recall, this amounts to computing $(\hat{a}, \hat{b}) = \arg\max_{(a,b)\in\mathbf{R}^{>0}\times\mathbf{R}^{>0}} p(\mathcal{D} \mid a, b)$.) This will require solving an optimization problem, for which you are free to use any optimization software you like (perhaps scipy.optimize would be useful). The empirical Bayes prior is then Beta$(\hat{a}, \hat{b})$, where $\hat{a}$ and $\hat{b}$ are our ML-2 estimates. Give the corresponding prior mean and standard deviation for this prior.
Solution:
The $\hat{a}, \hat{b}$ does not converge to a particular value!

```
In [73]: clicks = [50,160,180,0,0,1]
         imp = [10000,20000,60000,100,5,2]
```

```
In [94]: def empbayes_click(ab,click,impressions):
             prob = 1
             a = ab[0]
             b = ab[1]
             for i in range(len(click)):
                 top = special.beta(a+click[i],b+impressions[i] - click[i])
                 bot = special.beta(a,b)
                 com = top/float(bot)
                 prob = prob * com
             return prob
```

```
In [98]: initial_guess = [1,1]
         result = minimize(empbayes_click, initial_guess,args=(clicks,imp))
         result
```

```
Out[98]:       fun: 0.0
         hess_inv: array([[1, 0],
                [0, 1]])
              jac: array([0., 0.])
          message: 'Optimization terminated successfully.'
             nfev: 4
              nit: 0
             njev: 1
           status: 0
          success: True
                x: array([1., 1.])
```

4. Complete the following table:

	NumClicks	NumImpressions	MLE	MAP	PosteriorMean	PosteriorSD
App 1	50	10000	0.5%			
App 2	160	20000	0.8%			
App 3	180	60000	0.3%			
App 4	0	100	0%			
App 5	0	5	0%			
App 6	1	2	50%			

Make sure to take a look at the PosteriorSD values and note which are big and which are small.

8.3 [Optional] Hierarchical Bayes

In Section 8.2 we managed to get empirical Bayes ML-II estimates for a and b by assuming we had data from multiple apps. However, we didn't really address the issue that ML-II, as a maximum likelihood method, is prone to overfitting if we don't have enough data (in this case, enough apps). Moreover, a true Bayesian would reject this approach, since we're using our data to determine our prior. If we don't have enough confidence to choose parameters for a and b without looking at the data, then the only proper Bayesian approach is to put another prior on the parameters a and b. If you are very uncertain about values for a and b, you could put priors on them that have high variance.

1. [Optional] Suppose P is the Beta(a, b) distribution. Conceptually, rather than putting priors on a and b, it's easier to reason about priors on the mean m and the variance v of P. If we parameterize P by its mean m and the variance v, give an expression for the density function Beta$(\theta; m, v)$. You are free to use the internet to get this expression – just be confident it's correct. [Hint: To derive this, you may find it convenient to write some expression in terms of $\eta = a + b$.]
Solution:
Let
$$m = \frac{a}{a+b}, \quad 1 - m = \frac{b}{a+b}$$
and
$$v = \frac{ab}{(a+b)^2(a+b+1)}$$
$$\eta = a + b$$
, we can rewrite v in terms of m and η as
$$v = \frac{ab}{(a+b)^2}(\frac{1}{a+b+1}) = m(1-m)(\frac{1}{\eta+1})$$
Solving for η,
$$\eta = \frac{m(1-m)}{v} - 1$$
Rewriting v again in terms of only a and m,
$$v = (\frac{a}{a+b})(\frac{b}{a+b})(\frac{1}{a+b+1})$$
$$= (m)(1-m)(\frac{1}{\frac{a}{m}+1})$$
Solving for a we get
$$a = m(\frac{m(1-m)}{v} - 1)) = m\eta$$
and from $(\eta = a + b)$,
$$b = \eta - a = \eta - m\eta = \eta(1-m)$$
Therefore, the density function in terms of mean and variance is
$$\text{Beta}(\theta; m, v) = \frac{1}{B(m\eta, \eta(1-m))}\theta^{m\eta-1}(1-\theta)^{\eta(1-m)-1}$$

2. [Optional] Suggest a prior distribution to put on m and v. [Hint: You might want to use one of the distribution families given in this lecture.
Solution:
Let's first look at m, the mean of a Beta distribution is given as $\frac{a}{a+b}$ which means $0 \leq m \leq 1$. Therefore, we can use Beta distribution as a prior for m because beta distribution supports on $(0, 1)$.
For v, the variance of Beta distribution, the range is $(0, \infty)$, because $a > 0, b > 0$. We can use Gamma distribution for the prior of v because it supports on $(0, \infty)$.

3. [Optional] Once we have our prior on m and v, we can go "full Bayesian" and compute posterior distributions on $\theta_1, \ldots, \theta_d$. However, these no longer have closed forms. We would have to use approximation techniques, typically either a Monte Carlo sampling approach or a variational method, which are beyond the scope of this course[8]. After observing the data $\mathcal{D}$, m and v will have some posterior distribution $p(m, v \mid \mathcal{D})$. We can approximate that distribution by a point mass at the mode of that distribution $(m_{\text{MAP}}, v_{\text{MAP}}) = \arg\max_{m,v} p(m, v \mid \mathcal{D})$. **Give expressions** for the posterior distribution $p(\theta_1, \ldots, \theta_d \mid \mathcal{D})$, with and without this approximation. You do not need to give any explicit expressions here. It's fine to have expressions like $p(\theta_1, \ldots, \theta_d \mid m, v)$ in your solution. Without the approximation, you will probably need some integrals. It's these integrals that we need sampling or variational approaches to approximate. While one can see this approach as a way to approximate the proper Bayesian approach, one could also be skeptical and say this is just another way to determine your prior from the data. The estimators $(m_{\text{MAP}}, v_{\text{MAP}})$ are often called **MAP-II estimators**, since they are MAP estimators at a higher level of the Bayesian hierarchy.
Solution:
From the previous problem, θ_i is iid, so we can just sample it using m and v as parameters. With approximation,

$$p(\theta_1, \ldots, \theta_d)|\mathcal{D}) = p(\theta_1, \ldots, \theta_d | m_{\text{MAP}}, v_{\text{MAP}})$$

Without approximation,

$$p(\theta_1, \ldots, \theta_d)|\mathcal{D}) = \int_{m=0}^{1} \int_{v=0}^{\infty} p(\theta_1, \ldots, \theta_d | m, v) p(mv|\mathcal{D}) dv dm$$

[8] If you're very ambitious, you could try out a package like PyStan to see what happens.